SIMATS ENGINEERING

SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES

CHENNAI – 602105

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 - Database Design and Connectivity

Course Code:	DSA0501	**Course Title:**	Query Processing for Data Science With Real Time Applications
CO Assessed:	CO2	**Assessment:**	**AT1 – Database Design and Connectivity**
Weightage:		**Total Marks:**	20 Marks
CO2 Statement: Use Python basics and SQL libraries to connect to databases SQLite, MySQL, PostgreSQL and perform CRUD operations like Create, Read, Update, Delete. (BTL6)			
Student Name:	SHABARISH N	**Reg. No.:**	192424007
Section / Batch:	SLOT A /3RD YEAR	**Date of Submission:**	29-07-2026
Faculty In charge:	K Veena devi	**Project Mode:**	Individual Submission

Code of Conduct

I SHABARISH N (192424007) (Name / Reg No) certify that this submission is my original work and that I have adhered to all guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature: N. shabarish

Requirement Analysis (4 Marks)	ER Diagram Design (4 Marks)	Table Design & Normalization (4 Marks)	Database Connectivity using Python (4 Marks)	CRUD Operations (4 Marks)	Total (20 Marks)

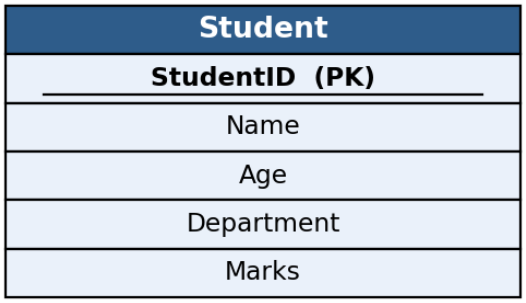
Question 1. Student Database Design and Connectivity

A college wants to automate the management of student records.

1. Requirement Analysis (4 Marks)

- The system must store, for every student: a unique identifier, name, age, department, and marks obtained.
- The system must allow new student records to be added.
- The system must allow all stored student records to be retrieved and displayed in a readable format.
- Each student must be uniquely identifiable, so StudentID is chosen as the primary key.

2. ER Diagram Design (4 Marks)



3. Table Design & Normalization (4 Marks)

```
CREATE TABLE IF NOT EXISTS Student (
    StudentID   INTEGER PRIMARY KEY AUTOINCREMENT,
    Name        TEXT NOT NULL,
    Age         INTEGER NOT NULL,
    Department  TEXT NOT NULL,
    Marks       REAL NOT NULL
);
```

The Student table is in 1NF (all attributes hold atomic, single values), 2NF (there are no composite keys, so no partial dependency is possible), and 3NF (Name, Age, Department, and Marks each depend only on the primary key StudentID and not on one another).

4. Database Connectivity using Python (4 Marks)

```
import sqlite3, os

DB = "db/q1_student.db"
if os.path.exists(DB):
    os.remove(DB)

conn = sqlite3.connect(DB)
cur = conn.cursor()

cur.execute("""
```

```
CREATE TABLE IF NOT EXISTS Student (
    StudentID   INTEGER PRIMARY KEY AUTOINCREMENT,
    Name        TEXT NOT NULL,
    Age         INTEGER NOT NULL,
    Department  TEXT NOT NULL,
    Marks       REAL NOT NULL
)
""")

students = [
    ("Aditi Sharma", 19, "Computer Science", 88.5),
    ("Rohan Mehta", 20, "Electronics", 76.0),
    ("Sneha Patil", 19, "Mechanical", 82.3),
    ("Kabir Singh", 21, "Civil", 69.8),
    ("Priya Nair", 20, "Computer Science", 91.2),
]

cur.executemany(
    "INSERT INTO Student (Name, Age, Department, Marks) VALUES (?, ?, ?, ?)",
    students
)
conn.commit()

print("Records inserted successfully.\n")
print("All Student Records:")
print(f"{'ID':<4}{'Name':<16}{'Age':<5}{'Department':<18}{'Marks':<6}")
print("-" * 49)
cur.execute("SELECT * FROM Student")
for row in cur.fetchall():
    print(f"{row[0]:<4}{row[1]:<16}{row[2]:<5}{row[3]:<18}{row[4]:<6}")

conn.close()
```

5. CRUD Operations Performed (4 Marks)

Create (table + records), Read (SELECT * to display all records).

Program Output

```
Records inserted successfully.

All Student Records:
ID  Name            Age  Department        Marks
-------------------------------------------------
1   Aditi Sharma    19   Computer Science  88.5
2   Rohan Mehta     20   Electronics       76.0
3   Sneha Patil     19   Mechanical        82.3
4   Kabir Singh     21   Civil             69.8
5   Priya Nair      20   Computer Science  91.2
```

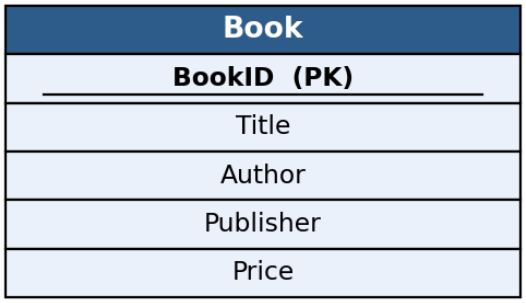
Question 2. Library Management Database

A library plans to maintain information about its books using an SQLite database.

1. Requirement Analysis (4 Marks)

- The system must store Book ID, Title, Author, Publisher, and Price for every book.
- The system must allow new book records to be inserted.
- The system must allow the price of a selected book to be updated.
- The system must allow the updated records to be retrieved and displayed.

2. ER Diagram Design (4 Marks)



3. Table Design & Normalization (4 Marks)

```
CREATE TABLE IF NOT EXISTS Book (
    BookID     INTEGER PRIMARY KEY AUTOINCREMENT,
    Title      TEXT NOT NULL,
    Author     TEXT NOT NULL,
    Publisher  TEXT NOT NULL,
    Price      REAL NOT NULL
);
```

The Book table satisfies 1NF, 2NF, and 3NF: every attribute is atomic, BookID is a single-column primary key (no partial dependency), and Title, Author, Publisher, and Price each depend directly on BookID only.

4. Database Connectivity using Python (4 Marks)

```
import sqlite3, os

DB = "db/q2_library.db"
if os.path.exists(DB):
    os.remove(DB)

conn = sqlite3.connect(DB)
cur = conn.cursor()

cur.execute("""
CREATE TABLE IF NOT EXISTS Book (
    BookID     INTEGER PRIMARY KEY AUTOINCREMENT,
    Title      TEXT NOT NULL,
    Author     TEXT NOT NULL,
    Publisher  TEXT NOT NULL,
    Price      REAL NOT NULL
)
""")

books = [
    ("Database System Concepts", "Silberschatz", "McGraw-Hill", 650.00),
    ("Python Crash Course", "Eric Matthes", "No Starch Press", 899.00),
    ("Clean Code", "Robert Martin", "Prentice Hall", 750.00),
    ("Operating System Concepts", "Silberschatz", "Wiley", 700.00),
```

```
    ("Introduction to Algorithms", "Cormen", "MIT Press", 1250.00),
]

cur.executemany(
    "INSERT INTO Book (Title, Author, Publisher, Price) VALUES (?, ?, ?, ?)",
    books
)
conn.commit()
print("Records inserted successfully.\n")

print("Updating price of 'Clean Code' to 699.00 ...")
cur.execute("UPDATE Book SET Price = ? WHERE Title = ?", (699.00, "Clean Code"))
conn.commit()
print("Update successful.\n")

print("Updated Book Records:")
print(f"{'ID':<4}{'Title':<28}{'Author':<16}{'Publisher':<18}{'Price':<8}")
print("-" * 74)
cur.execute("SELECT * FROM Book")
for row in cur.fetchall():
    print(f"{row[0]:<4}{row[1]:<28}{row[2]:<16}{row[3]:<18}{row[4]:<8}")

conn.close()
```

5. CRUD Operations Performed (4 Marks)

Create (table + records), Update (price of a selected book), Read (SELECT * to retrieve updated data).

Program Output

```
Records inserted successfully.

Updating price of 'Clean Code' to 699.00 ...
Update successful.

Updated Book Records:
ID  Title                       Author          Publisher         Price
--------------------------------------------------------------------------
1   Database System Concepts    Silberschatz    McGraw-Hill       650.0
2   Python Crash Course         Eric Matthes    No Starch Press   899.0
3   Clean Code                  Robert Martin   Prentice Hall     699.0
4   Operating System Concepts   Silberschatz    Wiley             700.0
5   Introduction to Algorithms  Cormen          MIT Press         1250.0
```

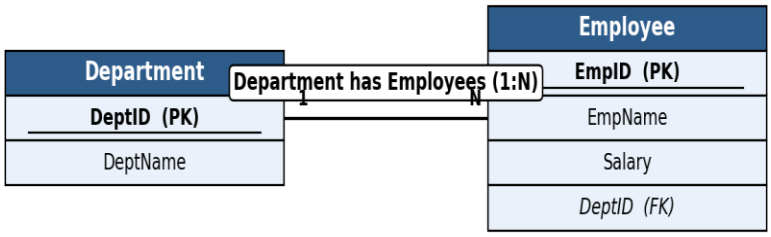
Question 3. Employee and Department Database Design

A company wants to store employee information and department details in an SQLite database.

1. Requirement Analysis (4 Marks)

- The system must store department details (DeptID, DeptName) and employee details (EmpID, EmpName, Salary).
- Every employee must belong to exactly one department (a department can have many employees) — a 1:N relationship.
- The system must allow both tables to be created and populated with sample data.
- The system must display each employee's name along with their department name using a JOIN.

2. ER Diagram Design (4 Marks)



3. Table Design & Normalization (4 Marks)

```
CREATE TABLE IF NOT EXISTS Department (
    DeptID    INTEGER PRIMARY KEY AUTOINCREMENT,
    DeptName  TEXT NOT NULL UNIQUE
);

CREATE TABLE IF NOT EXISTS Employee (
    EmpID     INTEGER PRIMARY KEY AUTOINCREMENT,
    EmpName   TEXT NOT NULL,
    Salary    REAL NOT NULL,
    DeptID    INTEGER NOT NULL,
    FOREIGN KEY (DeptID) REFERENCES Department(DeptID)
);
```

Both tables are in 3NF. Department and Employee data are separated into two tables to remove the repeating-group / transitive dependency that would occur if DeptName were stored directly inside Employee; DeptID (foreign key) links the two tables instead.

4. Database Connectivity using Python (4 Marks)

```
import sqlite3, os

DB = "db/q3_empdept.db"
if os.path.exists(DB):
    os.remove(DB)

conn = sqlite3.connect(DB)
cur = conn.cursor()
cur.execute("PRAGMA foreign_keys = ON")

cur.execute("""
CREATE TABLE IF NOT EXISTS Department (
    DeptID    INTEGER PRIMARY KEY AUTOINCREMENT,
    DeptName  TEXT NOT NULL UNIQUE
)
""")

cur.execute("""
CREATE TABLE IF NOT EXISTS Employee (
    EmpID     INTEGER PRIMARY KEY AUTOINCREMENT,
```

```
    EmpName   TEXT NOT NULL,
    Salary    REAL NOT NULL,
    DeptID    INTEGER NOT NULL,
    FOREIGN KEY (DeptID) REFERENCES Department(DeptID)
)
""")

departments = [("Computer Science",), ("Human Resources",), ("Finance",)]
cur.executemany("INSERT INTO Department (DeptName) VALUES (?)", departments)
conn.commit()

employees = [
    ("Amit Verma", 65000, 1),
    ("Neha Joshi", 58000, 2),
    ("Rahul Gupta", 72000, 1),
    ("Divya Kulkarni", 61000, 3),
    ("Suresh Iyer", 55000, 2),
]
cur.executemany(
    "INSERT INTO Employee (EmpName, Salary, DeptID) VALUES (?, ?, ?)",
    employees
)
conn.commit()
print("Records inserted successfully into Department and Employee tables.\n")

print("Employee Name with Corresponding Department (JOIN):")
print(f"{'EmpName':<18}{'Salary':<10}{'DeptName':<18}")
print("-" * 46)
cur.execute("""
SELECT Employee.EmpName, Employee.Salary, Department.DeptName
FROM Employee
JOIN Department ON Employee.DeptID = Department.DeptID
""")
for row in cur.fetchall():
    print(f"{row[0]:<18}{row[1]:<10}{row[2]:<18}")

conn.close()
```

5. CRUD Operations Performed (4 Marks)

Create (both tables + records), Read (JOIN query to display employee with department name).

Program Output

```
Records inserted successfully into Department and Employee tables.

Employee Name with Corresponding Department (JOIN):
EmpName           Salary    DeptName
----------------------------------------------
Amit Verma        65000.0   Computer Science
Neha Joshi        58000.0   Human Resources
Rahul Gupta       72000.0   Computer Science
Divya Kulkarni    61000.0   Finance
Suresh Iyer       55000.0   Human Resources
```

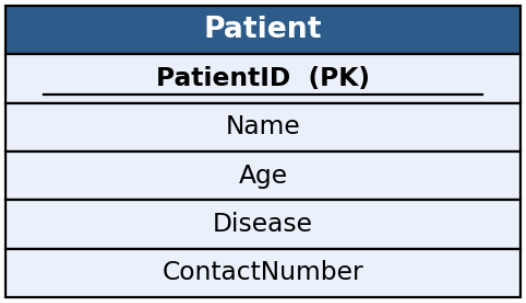
Question 4. Hospital Patient Database

A hospital requires a database to manage patient information.

1. Requirement Analysis (4 Marks)

- The system must store PatientID, Name, Age, Disease, and ContactNumber for every patient, plus a Discharged status flag.
- The system must allow new patient records to be inserted.
- The system must allow the contact number of a specified patient to be updated.
- The system must allow a discharged patient's record to be deleted, and the remaining records displayed.

2. ER Diagram Design (4 Marks)



3. Table Design & Normalization (4 Marks)

```
CREATE TABLE IF NOT EXISTS Patient (
    PatientID       INTEGER PRIMARY KEY AUTOINCREMENT,
    Name            TEXT NOT NULL,
    Age             INTEGER NOT NULL,
    Disease         TEXT NOT NULL,
    ContactNumber   TEXT NOT NULL,
    Discharged      INTEGER NOT NULL DEFAULT 0
);
```

The Patient table is in 1NF/2NF/3NF: attributes are atomic, PatientID is a single-column key, and every non-key attribute (Name, Age, Disease, ContactNumber, Discharged) depends only on PatientID.

4. Database Connectivity using Python (4 Marks)

```
import sqlite3, os

DB = "db/q4_patient.db"
if os.path.exists(DB):
    os.remove(DB)

conn = sqlite3.connect(DB)
cur = conn.cursor()

cur.execute("""
CREATE TABLE IF NOT EXISTS Patient (
    PatientID       INTEGER PRIMARY KEY AUTOINCREMENT,
    Name            TEXT NOT NULL,
    Age             INTEGER NOT NULL,
    Disease         TEXT NOT NULL,
    ContactNumber   TEXT NOT NULL,
```

```
    Discharged      INTEGER NOT NULL DEFAULT 0
)
""")

patients = [
    ("Anjali Rao", 34, "Fever", "9876543210", 0),
    ("Vikram Desai", 45, "Diabetes", "9823456712", 0),
    ("Meera Kulkarni", 29, "Fracture", "9812345678", 1),
    ("Sanjay Bhat", 52, "Hypertension", "9834567891", 0),
    ("Pooja Shah", 38, "Asthma", "9845678123", 1),
]
cur.executemany(
    "INSERT INTO Patient (Name, Age, Disease, ContactNumber, Discharged) VALUES (?, ?, ?, ?, ?)",
    patients
)
conn.commit()
print("Records inserted successfully.\n")

print("Updating contact number of 'Vikram Desai' ...")
cur.execute("UPDATE Patient SET ContactNumber = ? WHERE Name = ?", ("9900112233", "Vikram Desai"))
conn.commit()
print("Update successful.\n")

print("Deleting discharged patient records (Discharged = 1) ...")
cur.execute("DELETE FROM Patient WHERE Discharged = 1")
conn.commit()
print("Delete successful.\n")

print("Remaining Patient Records:")
print(f"{'ID':<4}{'Name':<16}{'Age':<5}{'Disease':<14}{'Contact':<12}")
print("-" * 51)
cur.execute("SELECT PatientID, Name, Age, Disease, ContactNumber FROM Patient")
for row in cur.fetchall():
    print(f"{row[0]:<4}{row[1]:<16}{row[2]:<5}{row[3]:<14}{row[4]:<12}")

conn.close()
```

5. CRUD Operations Performed (4 Marks)

Create (table + records), Update (contact number), Delete (discharged patient), Read (display remaining records).

Program Output

```
Records inserted successfully.

Updating contact number of 'Vikram Desai' ...
Update successful.

Deleting discharged patient records (Discharged = 1) ...
Delete successful.

Remaining Patient Records:
ID  Name            Age  Disease       Contact
---------------------------------------------------
1   Anjali Rao      34   Fever         9876543210
2   Vikram Desai    45   Diabetes      9900112233
4   Sanjay Bhat     52   Hypertension  9834567891
```

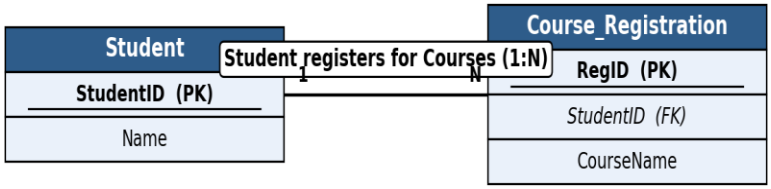
Question 5. Online Course Registration System

An online learning platform needs to maintain information about students and the courses they have registered for.

1. Requirement Analysis (4 Marks)

- The system must store student details (StudentID, Name).
- The system must record each course registration (RegID, StudentID, CourseName) — one student can register for many courses (1:N).
- The system must allow both tables to be created and populated.
- The system must display each student's name along with the courses they registered for using a JOIN.

2. ER Diagram Design (4 Marks)



3. Table Design & Normalization (4 Marks)

```
CREATE TABLE IF NOT EXISTS Student (
    StudentID   INTEGER PRIMARY KEY AUTOINCREMENT,
    Name        TEXT NOT NULL
);

CREATE TABLE IF NOT EXISTS Course_Registration (
    RegID       INTEGER PRIMARY KEY AUTOINCREMENT,
    StudentID   INTEGER NOT NULL,
    CourseName  TEXT NOT NULL,
    FOREIGN KEY (StudentID) REFERENCES Student(StudentID)
);
```

The design is in 3NF: a separate Course_Registration table removes the repeating group that would occur if a student could register for multiple courses inside a single Student row, avoiding data redundancy.

4. Database Connectivity using Python (4 Marks)

```
import sqlite3, os

DB = "db/q5_courseReg.db"
if os.path.exists(DB):
    os.remove(DB)

conn = sqlite3.connect(DB)
cur = conn.cursor()
```

```
cur.execute("PRAGMA foreign_keys = ON")

cur.execute("""
CREATE TABLE IF NOT EXISTS Student (
    StudentID   INTEGER PRIMARY KEY AUTOINCREMENT,
    Name        TEXT NOT NULL
)
""")

cur.execute("""
CREATE TABLE IF NOT EXISTS Course_Registration (
    RegID       INTEGER PRIMARY KEY AUTOINCREMENT,
    StudentID   INTEGER NOT NULL,
    CourseName  TEXT NOT NULL,
    FOREIGN KEY (StudentID) REFERENCES Student(StudentID)
)
""")

students = [("Ishaan Kapoor",), ("Tanvi Deshmukh",), ("Arjun Malhotra",)]
cur.executemany("INSERT INTO Student (Name) VALUES (?)", students)
conn.commit()

registrations = [
    (1, "Data Structures"),
    (1, "Web Development"),
    (2, "Machine Learning"),
    (3, "Database Systems"),
    (3, "Cloud Computing"),
]
cur.executemany(
    "INSERT INTO Course_Registration (StudentID, CourseName) VALUES (?, ?)",
    registrations
)
conn.commit()
print("Records inserted successfully into Student and Course_Registration tables.\n")

print("Students with Registered Courses (JOIN):")
print(f"{'StudentName':<18}{'CourseName':<20}")
print("-" * 38)
cur.execute("""
SELECT Student.Name, Course_Registration.CourseName
FROM Student
JOIN Course_Registration ON Student.StudentID = Course_Registration.StudentID
""")
for row in cur.fetchall():
    print(f"{row[0]:<18}{row[1]:<20}")

conn.close()
```

5. CRUD Operations Performed (4 Marks)

Create (both tables + records), Read (JOIN query showing student name with registered courses).

Program Output

```
Records inserted successfully into Student and Course_Registration tables.

Students with Registered Courses (JOIN):
StudentName       CourseName
--------------------------------------
```

```
Ishaan Kapoor     Data Structures
Ishaan Kapoor     Web Development
Tanvi Deshmukh    Machine Learning
Arjun Malhotra    Database Systems
Arjun Malhotra    Cloud Computing
```

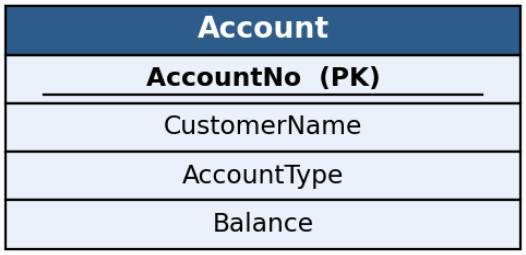
Question 6. Banking Account Management System

A bank wants to maintain customer account information using an SQLite database.

1. Requirement Analysis (4 Marks)

- The system must store AccountNo, CustomerName, AccountType, and Balance for every account.
- AccountType must be constrained to valid values (Savings/Current).
- The system must allow account details to be inserted.
- The system must allow the balance of a specified customer to be updated, and all records displayed.

2. ER Diagram Design (4 Marks)



3. Table Design & Normalization (4 Marks)

```
CREATE TABLE IF NOT EXISTS Account (
    AccountNo    INTEGER PRIMARY KEY AUTOINCREMENT,
    CustomerName TEXT NOT NULL,
    AccountType  TEXT NOT NULL CHECK (AccountType IN ('Savings','Current')),
    Balance      REAL NOT NULL DEFAULT 0
);
```

The Account table is in 3NF: all attributes are atomic, AccountNo is the single-column primary key, and CustomerName, AccountType, and Balance each depend only on AccountNo.

4. Database Connectivity using Python (4 Marks)

```
import sqlite3, os

DB = "db/q6_bank.db"
if os.path.exists(DB):
    os.remove(DB)

conn = sqlite3.connect(DB)
```

```
cur = conn.cursor()

cur.execute("""
CREATE TABLE IF NOT EXISTS Account (
    AccountNo    INTEGER PRIMARY KEY AUTOINCREMENT,
    CustomerName TEXT NOT NULL,
    AccountType  TEXT NOT NULL CHECK (AccountType IN ('Savings','Current')),
    Balance      REAL NOT NULL DEFAULT 0
)
""")

accounts = [
    ("Karan Malhotra", "Savings", 25000.00),
    ("Ritu Agarwal", "Current", 78000.00),
    ("Manish Kumar", "Savings", 15000.00),
    ("Swati Joshi", "Savings", 42000.00),
    ("Deepak Chawla", "Current", 96000.00),
]
cur.executemany(
    "INSERT INTO Account (CustomerName, AccountType, Balance) VALUES (?, ?, ?)",
    accounts
)
conn.commit()
print("Records inserted successfully.\n")

print("Updating balance for 'Manish Kumar' (deposit of 5000) ...")
cur.execute("UPDATE Account SET Balance = Balance + 5000 WHERE CustomerName = ?", ("Manish
Kumar",))
conn.commit()
print("Update successful.\n")

print("All Customer Account Records:")
print(f"{'AccNo':<7}{'CustomerName':<18}{'Type':<10}{'Balance':<10}")
print("-" * 45)
cur.execute("SELECT * FROM Account")
for row in cur.fetchall():
    print(f"{row[0]:<7}{row[1]:<18}{row[2]:<10}{row[3]:<10}")

conn.close()
```

5. CRUD Operations Performed (4 Marks)

Create (table + records), Update (account balance), Read (display all customer records).

Program Output

```
Records inserted successfully.

Updating balance for 'Manish Kumar' (deposit of 5000) ...
Update successful.

All Customer Account Records:
AccNo  CustomerName      Type      Balance
---------------------------------------------
1      Karan Malhotra    Savings   25000.0
2      Ritu Agarwal      Current   78000.0
3      Manish Kumar      Savings   20000.0
4      Swati Joshi       Savings   42000.0
5      Deepak Chawla     Current   96000.0
```

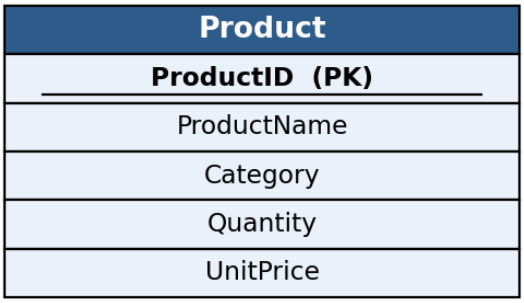
Question 7. Inventory Management System

A retail store needs to maintain product inventory using SQLite.

1. Requirement Analysis (4 Marks)

- The system must store ProductID, ProductName, Category, Quantity, and UnitPrice for every product.
- The system must allow sample products to be inserted.
- The system must identify products whose quantity is below a threshold (10 units).
- The system must allow the stock quantity of such products to be updated, and the updated inventory displayed.

2. ER Diagram Design (4 Marks)



3. Table Design & Normalization (4 Marks)

```
CREATE TABLE IF NOT EXISTS Product (
    ProductID    INTEGER PRIMARY KEY AUTOINCREMENT,
    ProductName  TEXT NOT NULL,
    Category     TEXT NOT NULL,
    Quantity     INTEGER NOT NULL,
    UnitPrice    REAL NOT NULL
);
```

The Product table is in 3NF: ProductID uniquely identifies each row, and ProductName, Category, Quantity, and UnitPrice all depend directly on ProductID with no transitive dependency between them.

4. Database Connectivity using Python (4 Marks)

```
import sqlite3, os

DB = "db/q7_inventory.db"
if os.path.exists(DB):
    os.remove(DB)

conn = sqlite3.connect(DB)
cur = conn.cursor()

cur.execute("""
```

```
CREATE TABLE IF NOT EXISTS Product (
    ProductID    INTEGER PRIMARY KEY AUTOINCREMENT,
    ProductName  TEXT NOT NULL,
    Category     TEXT NOT NULL,
    Quantity     INTEGER NOT NULL,
    UnitPrice    REAL NOT NULL
)
""")

products = [
    ("Notebook", "Stationery", 8, 45.00),
    ("Pen", "Stationery", 50, 10.00),
    ("Desk Lamp", "Electronics", 5, 650.00),
    ("USB Cable", "Electronics", 25, 120.00),
    ("Water Bottle", "Household", 7, 150.00),
]
cur.executemany(
    "INSERT INTO Product (ProductName, Category, Quantity, UnitPrice) VALUES (?, ?, ?, ?)",
    products
)
conn.commit()
print("Records inserted successfully.\n")

print("Products with Quantity < 10 (low stock):")
cur.execute("SELECT ProductID, ProductName, Quantity FROM Product WHERE Quantity < 10")
low_stock = cur.fetchall()
for row in low_stock:
    print(f"  ID {row[0]}: {row[1]} (Qty: {row[2]})")

print("\nRestocking low-quantity products (adding 20 units each) ...")
cur.execute("UPDATE Product SET Quantity = Quantity + 20 WHERE Quantity < 10")
conn.commit()
print("Update successful.\n")

print("Updated Inventory:")
print(f"{'ID':<4}{'ProductName':<14}{'Category':<14}{'Qty':<6}{'UnitPrice':<10}")
print("-" * 48)
cur.execute("SELECT * FROM Product")
for row in cur.fetchall():
    print(f"{row[0]:<4}{row[1]:<14}{row[2]:<14}{row[3]:<6}{row[4]:<10}")

conn.close()
```

5. CRUD Operations Performed (4 Marks)

Create (table + records), Read (filter low-stock products), Update (restock quantity), Read (display updated inventory).

Program Output

```
Records inserted successfully.

Products with Quantity < 10 (low stock):
  ID 1: Notebook (Qty: 8)
  ID 3: Desk Lamp (Qty: 5)
  ID 5: Water Bottle (Qty: 7)

Restocking low-quantity products (adding 20 units each) ...
Update successful.
```

```
Updated Inventory:
ID  ProductName   Category      Qty   UnitPrice
-----------------------------------------------
1   Notebook      Stationery    28    45.0
2   Pen           Stationery    50    10.0
3   Desk Lamp     Electronics   25    650.0
4   USB Cable     Electronics   25    120.0
5   Water Bottle  Household     27    150.0
```

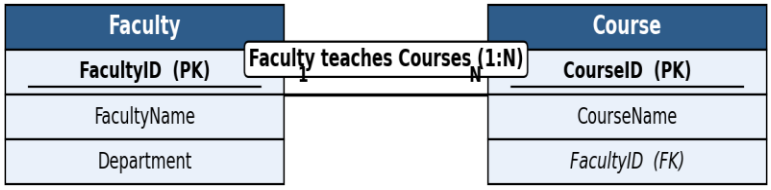
Question 8. Course and Faculty Management System

A university wants to maintain details of faculty members and the courses they teach.

1. Requirement Analysis (4 Marks)

- The system must store faculty details (FacultyID, FacultyName, Department).
- The system must store course details (CourseID, CourseName) and the faculty member teaching it — one faculty member can teach many courses (1:N).
- The system must allow both tables to be created and populated with sample data.
- The system must display each faculty member along with the courses assigned using a JOIN.

2. ER Diagram Design (4 Marks)



3. Table Design & Normalization (4 Marks)

```
CREATE TABLE IF NOT EXISTS Faculty (
    FacultyID    INTEGER PRIMARY KEY AUTOINCREMENT,
    FacultyName  TEXT NOT NULL,
    Department   TEXT NOT NULL
);

CREATE TABLE IF NOT EXISTS Course (
    CourseID     INTEGER PRIMARY KEY AUTOINCREMENT,
    CourseName   TEXT NOT NULL,
    FacultyID    INTEGER NOT NULL,
    FOREIGN KEY (FacultyID) REFERENCES Faculty(FacultyID)
);
```

Both tables are in 3NF. Splitting Faculty and Course into separate tables removes the repeating group that would occur if all of a faculty member's courses were stored inside a single Faculty row.

4. Database Connectivity using Python (4 Marks)

```
import sqlite3, os

DB = "db/q8_facultyCourse.db"
if os.path.exists(DB):
    os.remove(DB)

conn = sqlite3.connect(DB)
cur = conn.cursor()
cur.execute("PRAGMA foreign_keys = ON")

cur.execute("""
CREATE TABLE IF NOT EXISTS Faculty (
    FacultyID    INTEGER PRIMARY KEY AUTOINCREMENT,
    FacultyName  TEXT NOT NULL,
    Department   TEXT NOT NULL
)
""")

cur.execute("""
CREATE TABLE IF NOT EXISTS Course (
    CourseID     INTEGER PRIMARY KEY AUTOINCREMENT,
    CourseName   TEXT NOT NULL,
    FacultyID    INTEGER NOT NULL,
    FOREIGN KEY (FacultyID) REFERENCES Faculty(FacultyID)
)
""")

faculty = [
    ("Dr. Sunita Rao", "Computer Science"),
    ("Dr. Ashok Menon", "Mathematics"),
    ("Dr. Leela Krishnan", "Computer Science"),
]
cur.executemany("INSERT INTO Faculty (FacultyName, Department) VALUES (?, ?)", faculty)
conn.commit()

courses = [
    ("Database Management Systems", 1),
    ("Discrete Mathematics", 2),
    ("Operating Systems", 3),
    ("Linear Algebra", 2),
    ("Computer Networks", 1),
]
cur.executemany("INSERT INTO Course (CourseName, FacultyID) VALUES (?, ?)", courses)
conn.commit()
print("Records inserted successfully into Faculty and Course tables.\n")

print("Faculty with Assigned Courses (JOIN):")
print(f"{'FacultyName':<20}{'CourseName':<28}")
print("-" * 48)
cur.execute("""
SELECT Faculty.FacultyName, Course.CourseName
FROM Faculty
JOIN Course ON Faculty.FacultyID = Course.FacultyID
ORDER BY Faculty.FacultyName
""")
for row in cur.fetchall():
    print(f"{row[0]:<20}{row[1]:<28}")

conn.close()
```

5. CRUD Operations Performed (4 Marks)

Create (both tables + records), Read (JOIN query showing faculty with assigned courses).

Program Output

```
Records inserted successfully into Faculty and Course tables.

Faculty with Assigned Courses (JOIN):
FacultyName         CourseName
-----------------------------------------------
Dr. Ashok Menon     Discrete Mathematics
Dr. Ashok Menon     Linear Algebra
Dr. Leela Krishnan  Operating Systems
Dr. Sunita Rao      Database Management Systems
Dr. Sunita Rao      Computer Networks
```

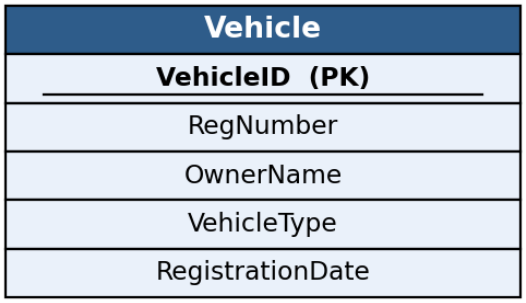
Question 9. Vehicle Registration Database

A transport department plans to maintain vehicle registration details in an SQLite database.

1. Requirement Analysis (4 Marks)

- The system must store VehicleID, RegNumber, OwnerName, VehicleType, and RegistrationDate for every vehicle.
- Each registration number must be unique.
- The system must allow vehicle records to be inserted.
- The system must allow a vehicle to be searched for using its registration number, and its details displayed.

2. ER Diagram Design (4 Marks)



3. Table Design & Normalization (4 Marks)

```
CREATE TABLE IF NOT EXISTS Vehicle (
    VehicleID        INTEGER PRIMARY KEY AUTOINCREMENT,
    RegNumber        TEXT NOT NULL UNIQUE,
    OwnerName        TEXT NOT NULL,
    VehicleType      TEXT NOT NULL,
    RegistrationDate TEXT NOT NULL
);
```

The Vehicle table is in 3NF: VehicleID is the primary key, RegNumber is a unique candidate key, and all remaining attributes depend only on VehicleID with no inter-attribute dependency.

4. Database Connectivity using Python (4 Marks)

```
import sqlite3, os

DB = "db/q9_vehicle.db"
if os.path.exists(DB):
    os.remove(DB)

conn = sqlite3.connect(DB)
cur = conn.cursor()

cur.execute("""
CREATE TABLE IF NOT EXISTS Vehicle (
    VehicleID       INTEGER PRIMARY KEY AUTOINCREMENT,
    RegNumber       TEXT NOT NULL UNIQUE,
    OwnerName       TEXT NOT NULL,
    VehicleType     TEXT NOT NULL,
    RegistrationDate TEXT NOT NULL
)
""")

vehicles = [
    ("MH12AB1234", "Nikhil Rane", "Car", "2022-05-10"),
    ("MH14CD5678", "Sarita Pillai", "Motorcycle", "2021-11-23"),
    ("MH04EF9012", "Rajesh Nair", "Car", "2023-02-15"),
    ("MH20GH3456", "Vaishali Kale", "Truck", "2020-08-30"),
    ("MH31IJ7890", "Farhan Sheikh", "Motorcycle", "2023-07-01"),
]
cur.executemany(
    "INSERT INTO Vehicle (RegNumber, OwnerName, VehicleType, RegistrationDate) VALUES (?, ?, ?,
?)",
    vehicles
)
conn.commit()
print("Records inserted successfully.\n")

search_reg = "MH04EF9012"
print(f"Searching for vehicle with Registration Number = {search_reg} ...\n")
cur.execute("SELECT * FROM Vehicle WHERE RegNumber = ?", (search_reg,))
result = cur.fetchone()
if result:
    print("Vehicle Found:")
    print(f"  VehicleID       : {result[0]}")
    print(f"  RegNumber       : {result[1]}")
    print(f"  OwnerName       : {result[2]}")
    print(f"  VehicleType     : {result[3]}")
    print(f"  RegistrationDate : {result[4]}")
else:
    print("No vehicle found with that registration number.")

conn.close()
```

5. CRUD Operations Performed (4 Marks)

Create (table + records), Read (search a vehicle by registration number and display it).

Program Output

```
Records inserted successfully.

Searching for vehicle with Registration Number = MH04EF9012 ...

Vehicle Found:
  VehicleID        : 3
  RegNumber        : MH04EF9012
  OwnerName        : Rajesh Nair
  VehicleType      : Car
  RegistrationDate : 2023-02-15
```

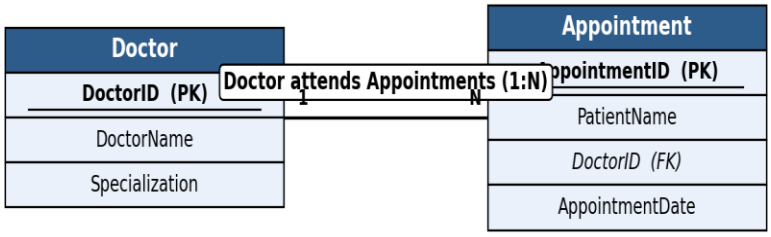
Question 10. Hospital Appointment Management System

A hospital maintains separate tables for Doctors and Appointments.

1. Requirement Analysis (4 Marks)

- The system must store doctor details (DoctorID, DoctorName, Specialization).
- The system must store appointment details (AppointmentID, PatientName, DoctorID, AppointmentDate) — one doctor can have many appointments (1:N).
- The system must allow both tables to be created and populated with sample data.
- The system must display the doctor name along with appointment details using a JOIN.

2. ER Diagram Design (4 Marks)



3. Table Design & Normalization (4 Marks)

```
CREATE TABLE IF NOT EXISTS Doctor (
    DoctorID       INTEGER PRIMARY KEY AUTOINCREMENT,
    DoctorName     TEXT NOT NULL,
    Specialization TEXT NOT NULL
);

CREATE TABLE IF NOT EXISTS Appointment (
    AppointmentID   INTEGER PRIMARY KEY AUTOINCREMENT,
    PatientName     TEXT NOT NULL,
    DoctorID        INTEGER NOT NULL,
    AppointmentDate TEXT NOT NULL,
    FOREIGN KEY (DoctorID) REFERENCES Doctor(DoctorID)
);
```

Both tables are in 3NF. Doctor and Appointment are kept separate so that doctor details are not repeated for every appointment row, and DoctorID (foreign key) links each appointment back to its doctor.

4. Database Connectivity using Python (4 Marks)

```
import sqlite3, os

DB = "db/q10_appointment.db"
if os.path.exists(DB):
    os.remove(DB)

conn = sqlite3.connect(DB)
cur = conn.cursor()
cur.execute("PRAGMA foreign_keys = ON")

cur.execute("""
CREATE TABLE IF NOT EXISTS Doctor (
    DoctorID       INTEGER PRIMARY KEY AUTOINCREMENT,
    DoctorName     TEXT NOT NULL,
    Specialization TEXT NOT NULL
)
""")

cur.execute("""
CREATE TABLE IF NOT EXISTS Appointment (
    AppointmentID   INTEGER PRIMARY KEY AUTOINCREMENT,
    PatientName     TEXT NOT NULL,
    DoctorID        INTEGER NOT NULL,
    AppointmentDate TEXT NOT NULL,
    FOREIGN KEY (DoctorID) REFERENCES Doctor(DoctorID)
)
""")

doctors = [
    ("Dr. Neha Kapadia", "Cardiologist"),
    ("Dr. Sameer Joshi", "Orthopedic"),
    ("Dr. Kavita Reddy", "Dermatologist"),
]
cur.executemany("INSERT INTO Doctor (DoctorName, Specialization) VALUES (?, ?)", doctors)
conn.commit()

appointments = [
    ("Ramesh Iyer", 1, "2026-08-01"),
    ("Sunita Bhosale", 2, "2026-08-02"),
    ("Farooq Ahmed", 1, "2026-08-03"),
    ("Geeta Menon", 3, "2026-08-04"),
    ("Ajay Thakur", 2, "2026-08-05"),
]
cur.executemany(
    "INSERT INTO Appointment (PatientName, DoctorID, AppointmentDate) VALUES (?, ?, ?)",
    appointments
)
conn.commit()
print("Records inserted successfully into Doctor and Appointment tables.\n")

print("Doctor Name with Appointment Details (JOIN):")
print(f"{'DoctorName':<20}{'PatientName':<18}{'AppointmentDate':<16}")
print("-" * 54)
cur.execute("""
SELECT Doctor.DoctorName, Appointment.PatientName, Appointment.AppointmentDate
FROM Doctor
JOIN Appointment ON Doctor.DoctorID = Appointment.DoctorID
```

```
ORDER BY Appointment.AppointmentDate
""")
for row in cur.fetchall():
    print(f"{row[0]:<20}{row[1]:<18}{row[2]:<16}")

conn.close()
```

5. CRUD Operations Performed (4 Marks)

Create (both tables + records), Read (JOIN query showing doctor name with appointment details).

Program Output

```
Records inserted successfully into Doctor and Appointment tables.

Doctor Name with Appointment Details (JOIN):
DoctorName          PatientName       AppointmentDate
-----------------------------------------------------
Dr. Neha Kapadia    Ramesh Iyer       2026-08-01
Dr. Sameer Joshi    Sunita Bhosale    2026-08-02
Dr. Neha Kapadia    Farooq Ahmed      2026-08-03
Dr. Kavita Reddy    Geeta Menon       2026-08-04
Dr. Sameer Joshi    Ajay Thakur       2026-08-05
```